

OOP2 KU

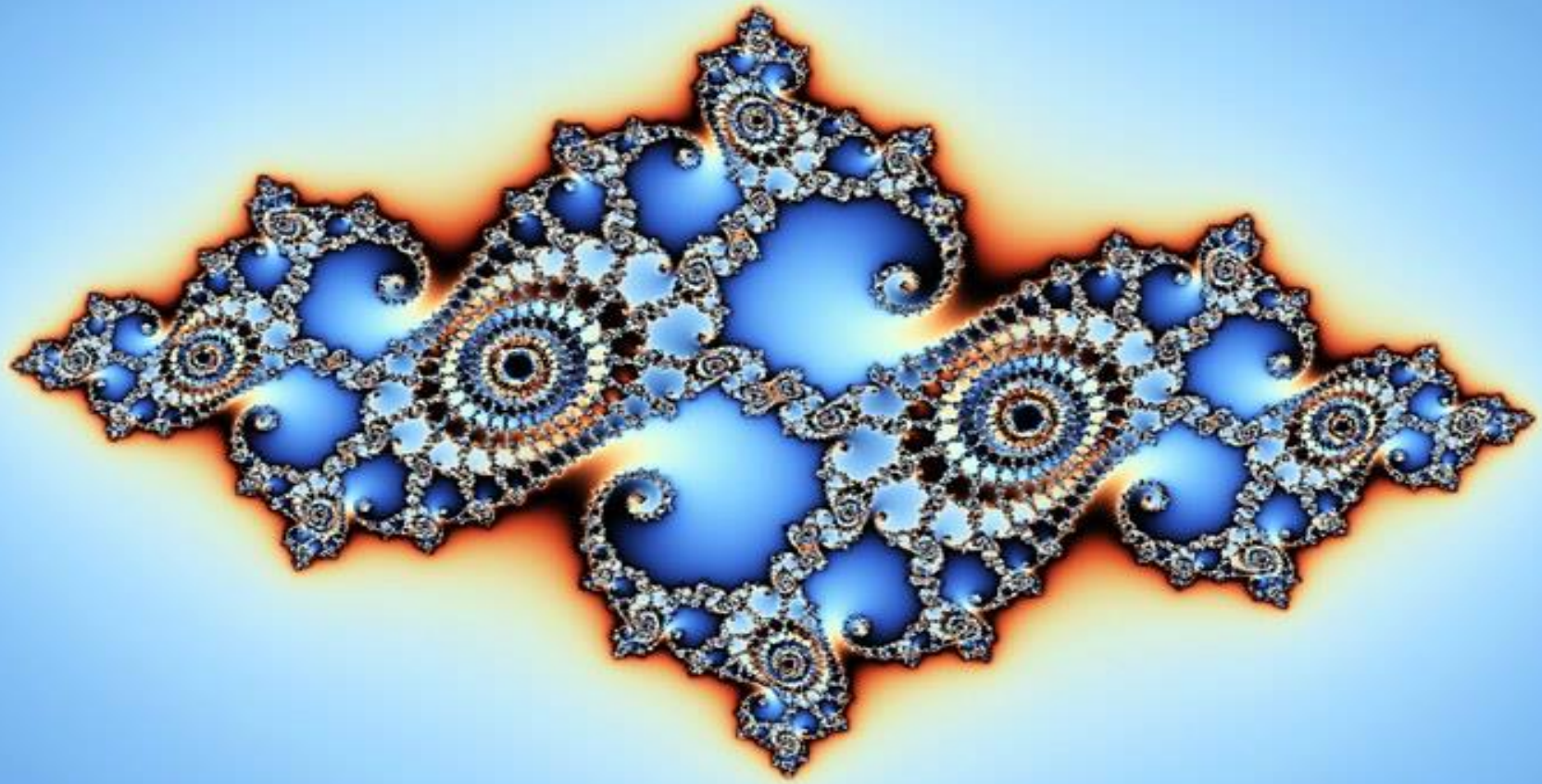
A Layman's Introduction to Fractals and the Bonus Assignment

Session X

Sebastian David Albu, WS 2024

Based on slides from WS 2021 and WS 2022

Institute of Computer Graphics and Knowledge Visualization (CGV)

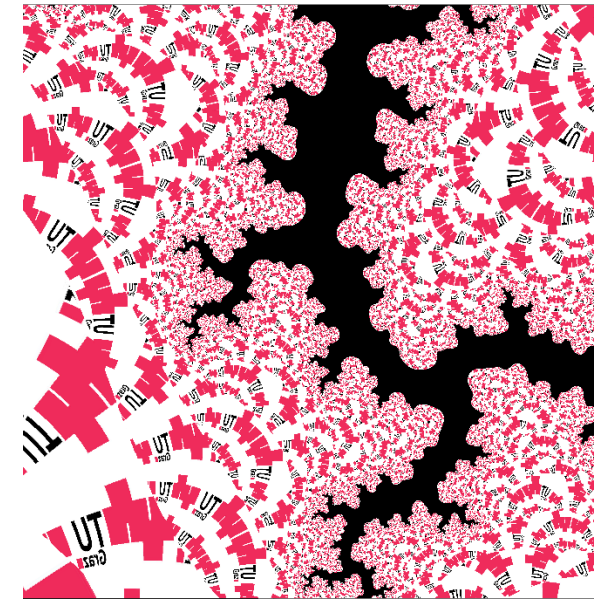
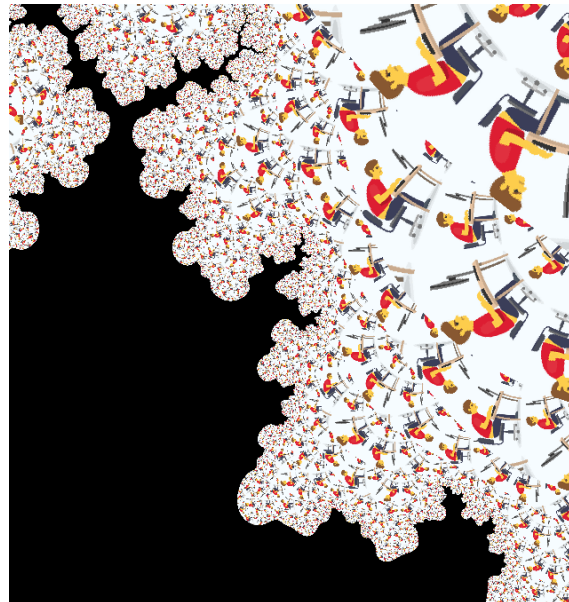


Up Front...

- This slide deck was created before the final task and points were fixed. Therefore here you find way more information than needed.
- For completing the current version of the bonus assignment the README and the slide deck shown in the lecture is MORE than enough.
- This slide deck here is simply uploaded for completeness and in case anyone is curious to read more into the documentation of the used functions in the framework

Up Front...

- Fractal sets will not be the main focus of exam interviews
- We will treat them very informally in this presentation
- They will just be a means to create interesting pictures procedurally
 - *Spoiler:*
very little code needed!
 - (Entire reference solution for bonus assignment has < 300 LOC) !!

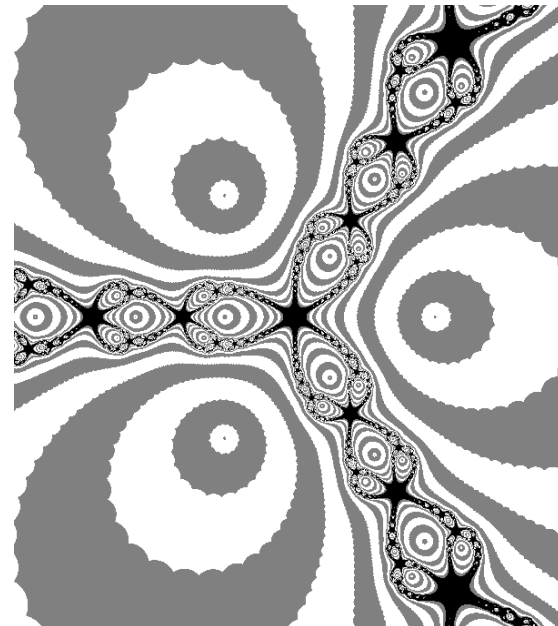
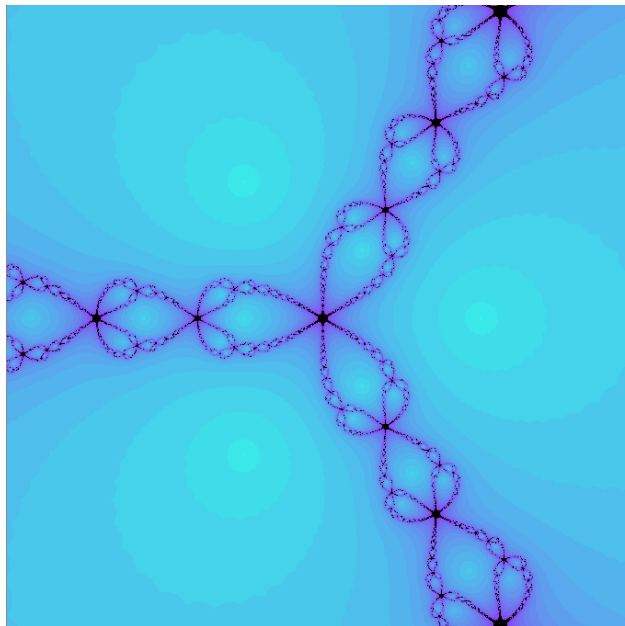
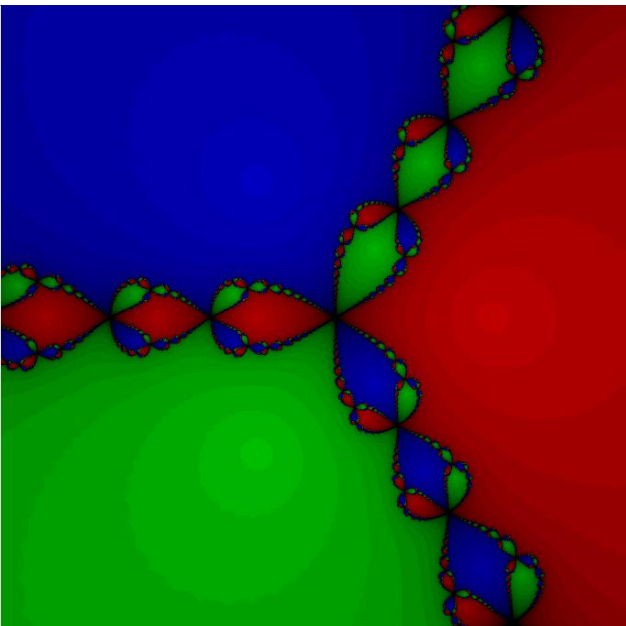


Structure of Presentation

- History and overview of fractals
- Math refresher (complex numbers)
- Brief introduction to GLSL vertex and fragment shaders
- Calculation of Mandelbrot set (with GLSL)
- Layout of the assignment and useful OpenGL functions in Java
- Guide for the bonus assignment
- Visualize calculations

Fractal Sets – Newton Raphson Fractal

- Named after Sir Isaac Newton and Joseph Raphson
- Great introduction to the world of fractals (see Link below)



Sir Isaac Newton



[This Photo](#) by Unknown Author is licensed under [CC BY-NC-ND](#)

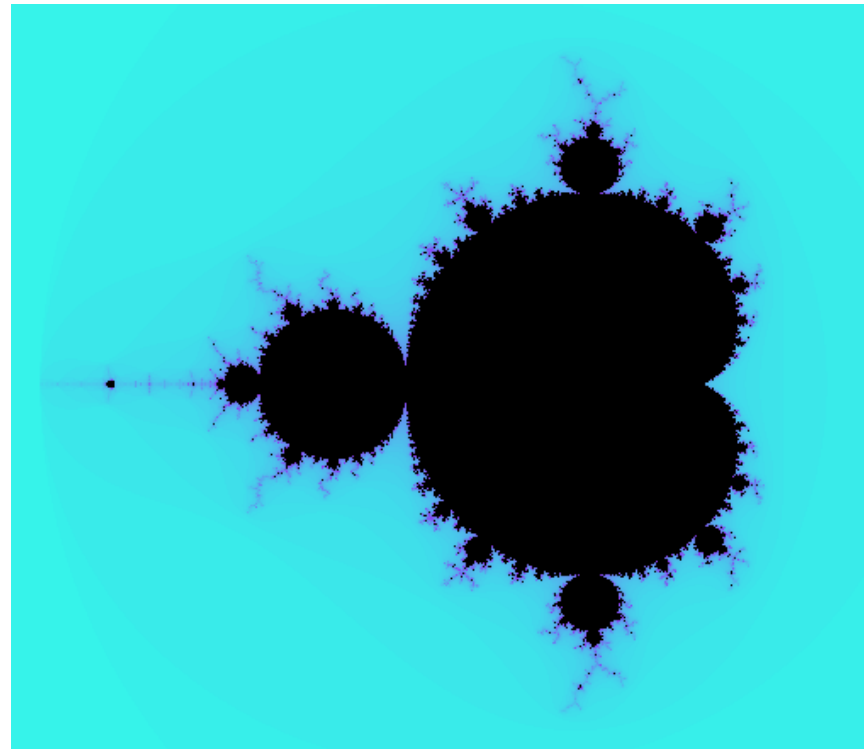
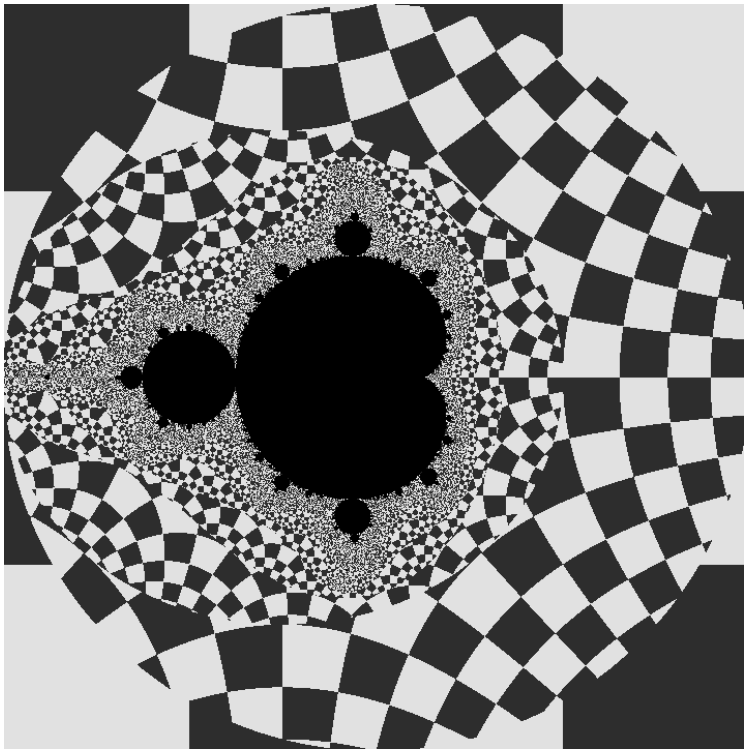
Recommended:

“Newton’s method, and the fractal it creates that Newton knew nothing about” by 3Blue1Brown

<https://www.youtube.com/watch?v=-RdOwhmqP5s>

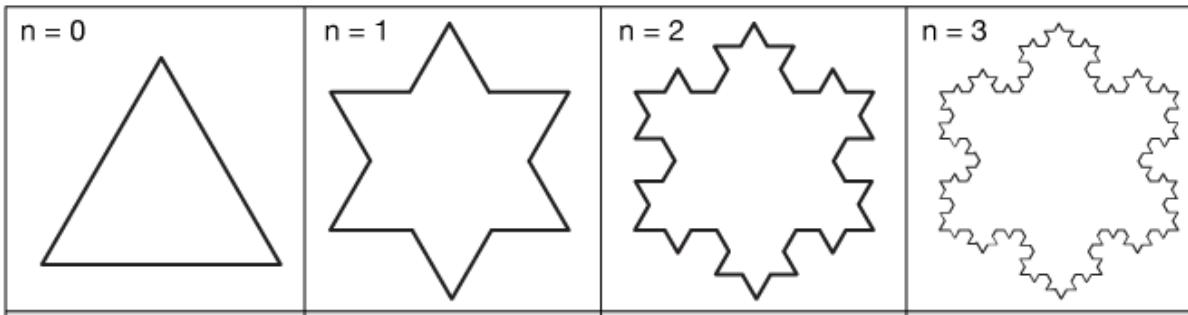
Fractal Sets – Mandelbrot Set

- Named after Benoit Mandelbrot

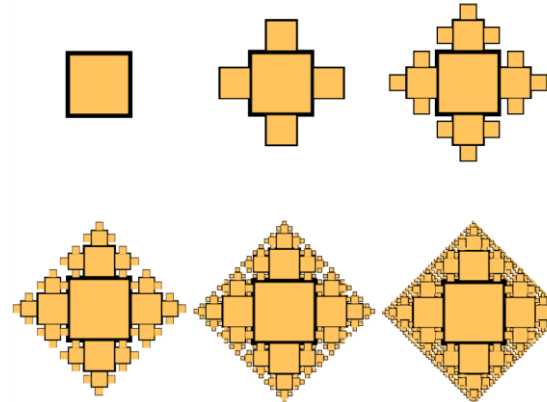


Benoit Mandelbrot
(1924-2010)

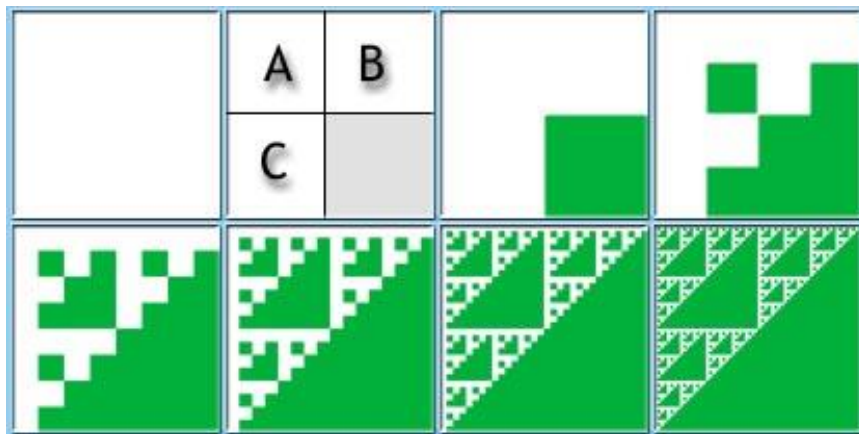
Fractal Curves & Iterated Function Systems



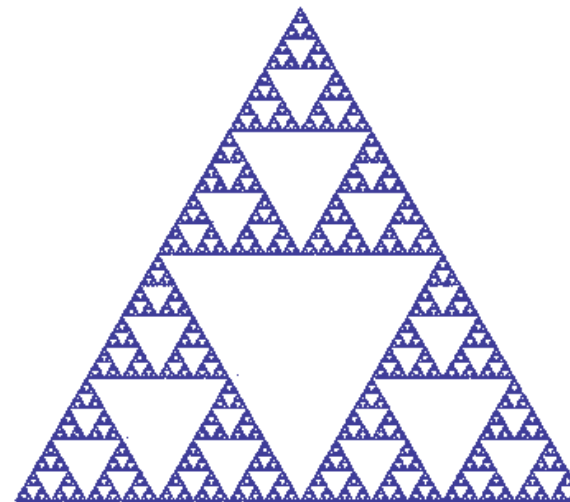
[This Photo](#) by Unknown Author is licensed under [CC BY-SA-NC](#)



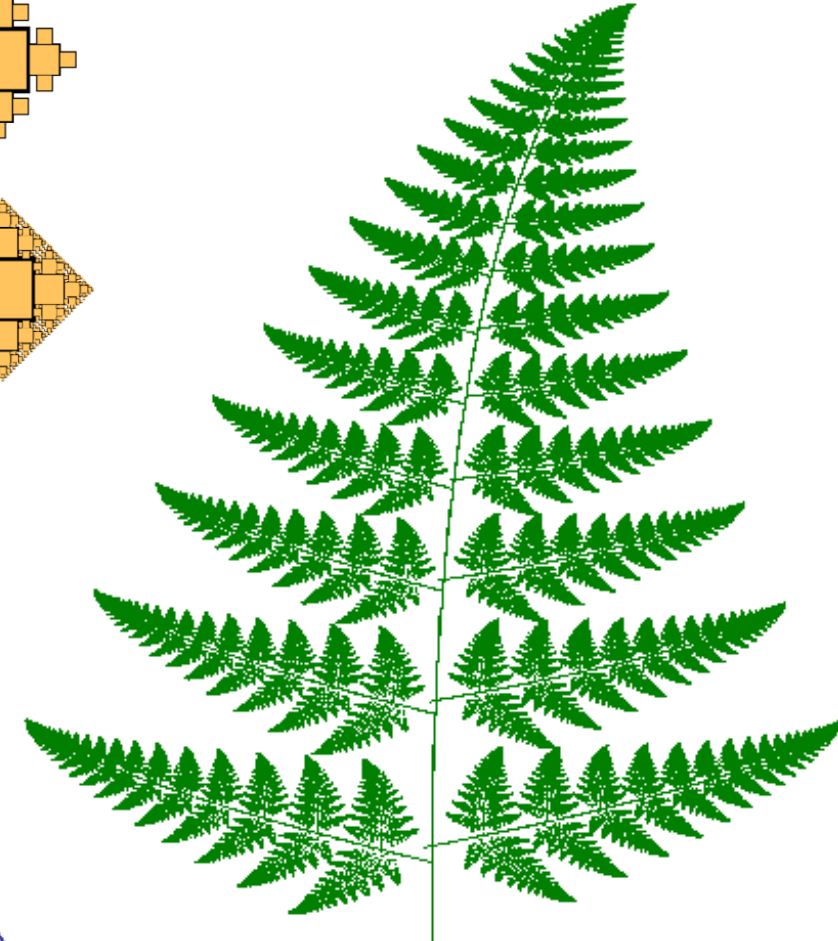
[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

Fractals used in the KU

- We will only calculate and display the Mandelbrot set
 - We will slightly modify the equations to keep things interesting
 - Details are given in assignment descriptions in GitLab repos
 - Slides only show the “base” fractal sets explicitly
- The fractal set is defined on the *complex plane*
 - Plane that contains all complex numbers
 - This will be defined in the vertex shader (**complexPlane.vert**)

Complex Numbers - Refresher

- Typically written as: $z = a + b * i$
- Consist of *real part* a and *imaginary part* b
 - Both parts are "standard" real numbers for example, $z = 69 + 420 * i$
- i is the imaginary unit, and $i^2 = -1$
 - (this means that we can calculate the square root of negative numbers)

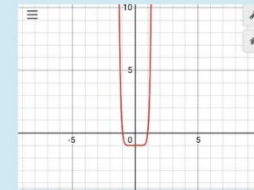
Complex Numbers - Refresher

- Let z_1 and z_2 be two complex numbers
 - $z_1 = a_1 + b_1i$
 - $z_2 = a_2 + b_2i$
- Adding complex numbers: add real parts and imaginary parts
 - $z_1 + z_2 = (a_1 + a_2) + (b_1 + b_2)i$
- Multiplying complex numbers
 - $z_1 * z_2 = (a_1a_2 - b_1b_2) + (a_1b_2 + a_2b_1)i$

Complex Numbers - Refresher

STOP USING COMPLEX NUMBERS

- Yes we get it, $i^2 = -1$. And?????
- "Trust me guys this order 10 polynomial has 10 roots" Like look at the graph is your brain malfunctioning???
- These are the ways people represent complex numbers:

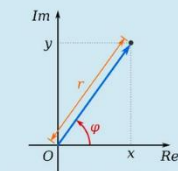


DOES THIS LOOK LIKE 10 ROOTS TO YOU?

$$i \equiv \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \quad |z| (\cos \theta + i \sin \theta)$$

A MATRIX???

YEAH THIS MAKES A LOT OF SENSE



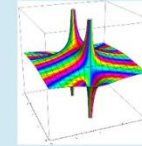
USING A GRAPH NOW???

$$z = re^{i\theta}$$

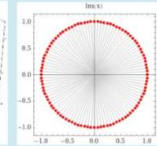
BLATANTLY STEALING
EULER'S FORMULA

$$a + ib$$

Ok this one actually
makes sense



HUH????



?????????

"Hi can I have $3i$ apples please"

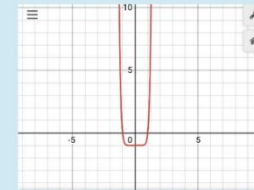
"I am  years old"

THEY HAVE PLAYED US FOR
ABSOLUTE FOOLS

Complex Numbers - Refresher

STOP USING COMPLEX NUMBERS

- Yes we get it, $i^2 = -1$. And?????
- "Trust me guys this order 10 polynomial has 10 roots" Like look at the graph is your brain malfunctioning???
- These are the ways people represent complex numbers:

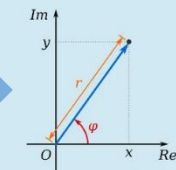


DOES THIS LOOK LIKE 10 ROOTS TO YOU?

$$i \equiv \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \quad |z| (\cos \theta + i \sin \theta)$$

A MATRIX???

YEAH THIS MAKES A LOT OF SENSE



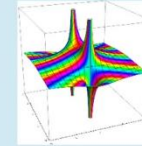
USING A GRAPH NOW???

$$z = re^{i\theta}$$

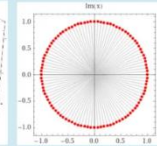
BLATANTLY STEALING
EULER'S FORMULA

$$a + ib$$

Ok this one actually
makes sense



HUH????



?????????

"Hi can I have $3i$ apples please"

"I am  years old"

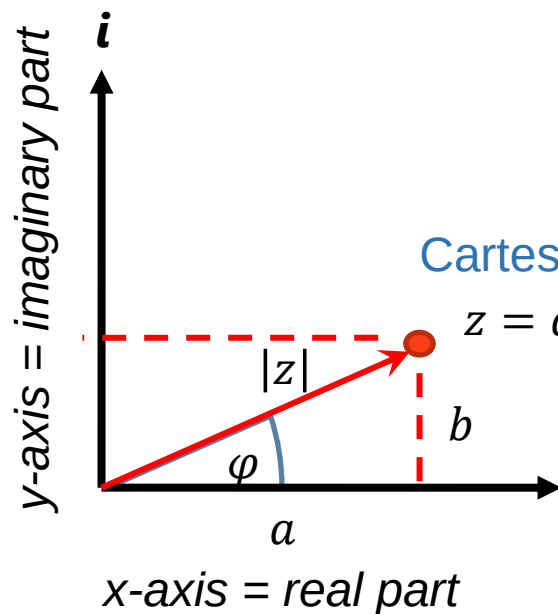
THEY HAVE PLAYED US FOR
ABSOLUTE FOOLS

Cartesian Form Polar Form

- We can think of a complex number as a 2D coordinate on the Cartesian complex plane

Polar Form: $|z|$
 $= \sqrt{a^2 + b^2}$

$$\varphi = \begin{cases} \tan^{-1}\left(\frac{b}{a}\right) \dots \text{if } a \geq 0 \\ \tan^{-1}\left(\frac{b}{a}\right) + \pi \dots \text{if } a < 0 \end{cases}$$



Cartesian Form:

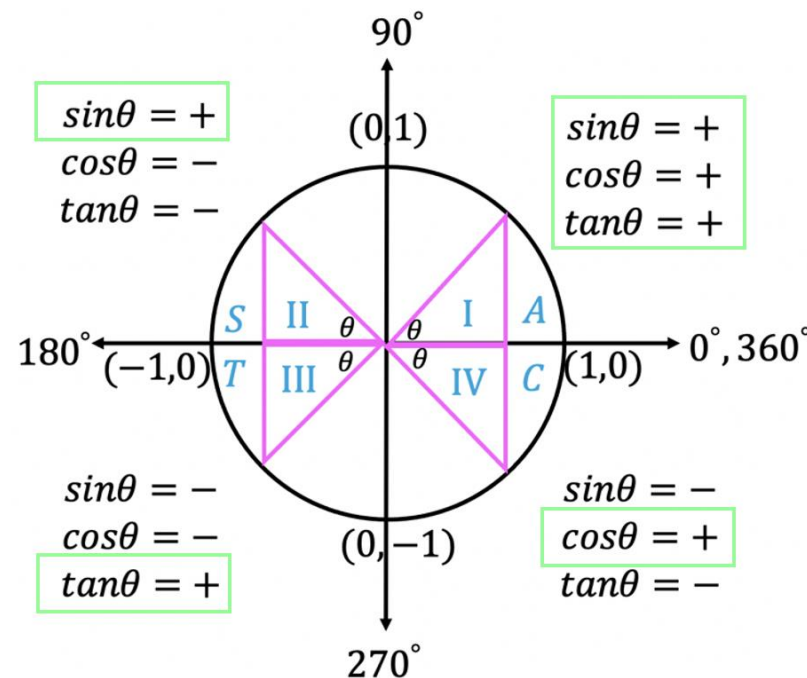
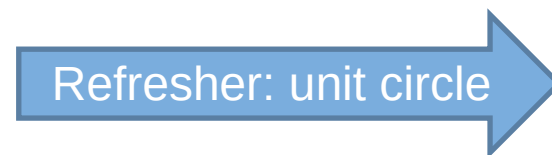
$$z = a + bi$$

Alternative ways
to calculate
(sign checks left as an
exercise)

$$\begin{cases} \cos(\varphi) = \frac{a}{|z|} \Leftrightarrow \varphi = \cos^{-1}\left(\frac{a}{|z|}\right) \\ \sin(\varphi) = \frac{b}{|z|} \Leftrightarrow \varphi = \sin^{-1}\left(\frac{b}{|z|}\right) \end{cases}$$

Polar Form – Pitfalls

- Pay attention to the quadrants the complex number lies in
 - Sanity check calculated polar-form angle
 - Hint: read up on `Math.atan2()`, and note seemingly “backward” parameter order...
- Know your units (DEG vs. RAD)!

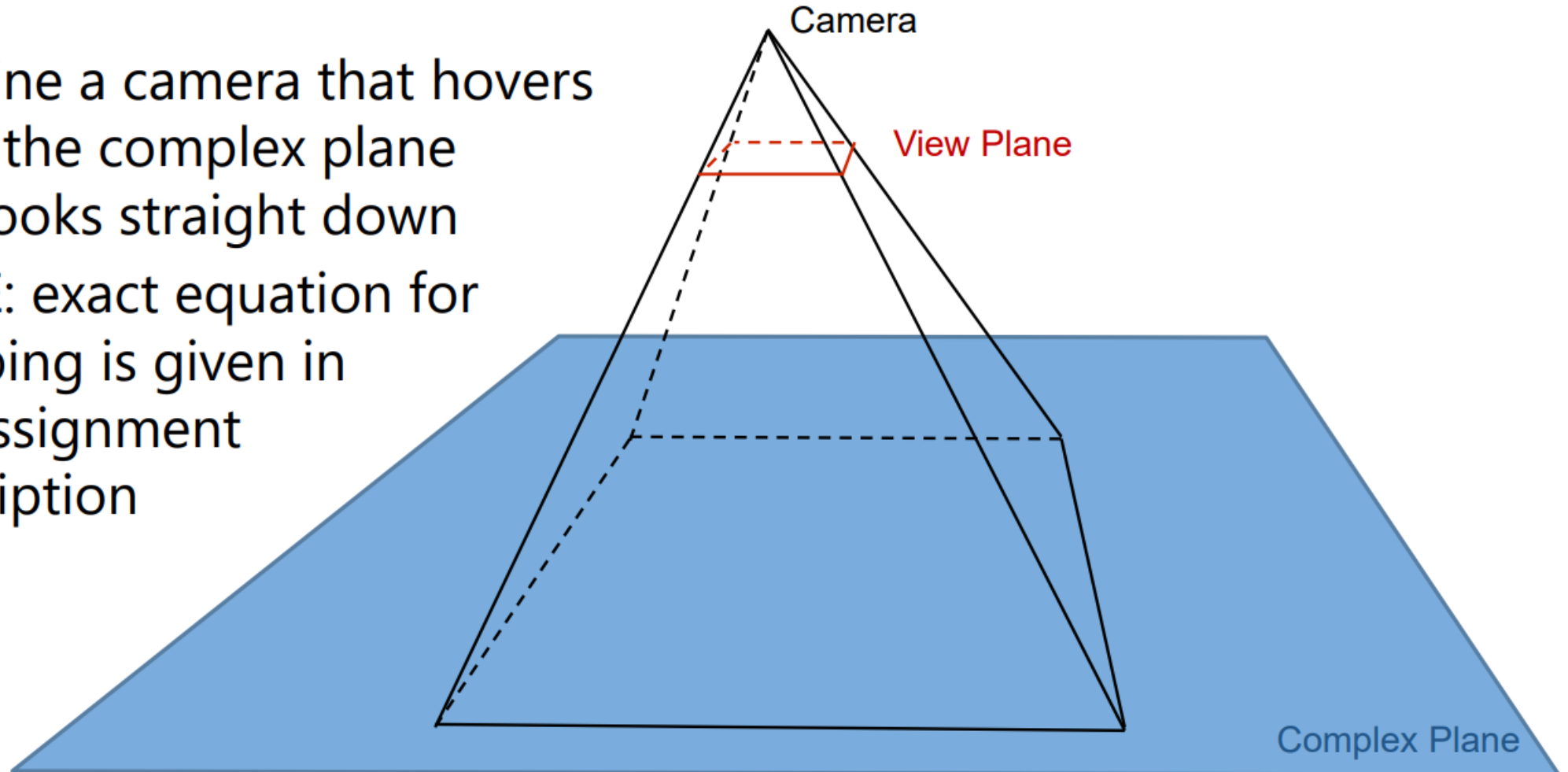


Polar Form vs. Cartesian Form

- Note that both cartesian form and polar form are 2D coordinates
- Cartesian form:
 - 2 real numbers that denote distances along the x-axis and the y-axis
- Polar form:
 - 2 real numbers that denote the magnitude (= Length) and the angle of the resulting point on the complex plane
 - Magnitude = distance to origin
 - Angle = measured from positive x-axis in counter-clockwise direction just as in unit-circle

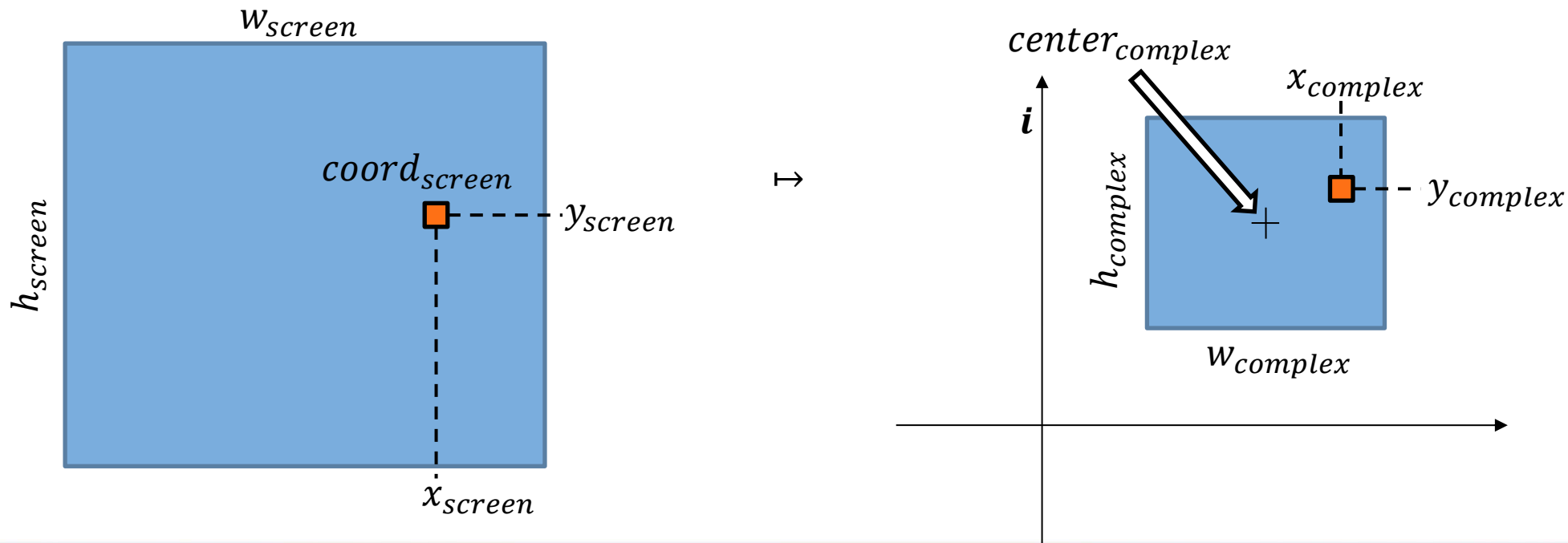
Mapping Pixel Coords to Complex Numbers

- Imagine a camera that hovers over the complex plane and looks straight down
- NOTE: exact equation for mapping is given in the assignment description



Mapping Pixel Coords to Complex Numbers

- Let w_{screen} and h_{screen} denote the screen size [in pixels]
- Map pixel $coord_{screen}$ to rectangular area of size $w_{complex}$ on the complex plane centered about the complex number $center_{complex}$, where the relative position of the pixel on the screen is preserved within rectangular area on complex plane



Vertex Shader of the Complex Plane

- Calculation in the **complexPlane.vert** file
 - Using GLSL syntax for both vertex and fragment shaders
 - This vertex shader is used to map the vertices of the complex plane to screen space.
 - It also passes the position of each vertex as a varying variable to the fragment shader.
- **Key Variables:**
 - **attribute vec2 position:** Represents the 2D position of each vertex in the complex plane.
 - **uniform mat4 projection:** A 4x4 matrix used to transform vertex coordinates from the complex plane to the screen space.
 - **varying vec2 c:** Passes the position of each vertex to the fragment shader for further processing.

Transformation and Output in Vertex Shader

- Calculation in the **complexPlane.vert** file
- Transformation:
 - Converts the 2D position into a 4D vector with a z value of 0 and a w (homogeneous coordinate) of 1.
 - Multiplies this vector by the projection matrix to transform it into screen coordinates.
- Output (Passing to Fragment Shader)
 - Directly assigns the vertex position to c, which will be used in the fragment shader to determine the color of the corresponding pixel.

Calculating the Mandelbrot Set

- Fragment Shader
 - This fragment shader is used in the rendering pipeline to calculate and visualize fractals, such as the Mandelbrot set.
 - This specific shader uses complex number arithmetic and iterative methods to determine the color of each pixel based on its position in the complex plane.
- → We first need the vertex shader for the complex plane

Calculating the Mandelbrot Set

- Fragment Shader – Key Variables of the **.frag** file
- Note that we use GLSL syntax
 - Uniforms:
 - **power**: Controls the exponentiation in the complex number calculations.
 - **maxIterations**: Determines the maximum number of iterations before a point is considered to belong to the Mandelbrot set.
 - Varying:
 - **c**: The complex coordinate corresponding to the current pixel being processed.

Calculating the Mandelbrot Set

- Complex Number Exponentiation
 - Note that we use GLSL syntax.
- **vec2 complexPow(in vec2 z, in float k)**
 - This function computes z^k where z is a complex number represented by a vec2.
 - Calculate the argument (angle) of the complex number using atan.
 - Calculate the magnitude (length) of the complex number. (GLSL: length(z))
 - Raise the magnitude to the power of k.
 - Multiply the argument by k.
 - Return the new complex number in polar coordinates, converting it back to cartesian form with cos and sin. → **mag * vec2(cos(angle), sin(angle))**

Calculating the Mandelbrot Set

- Mandelbrot Iteration
 - Note that we use GLSL syntax.
- Can be implemented directly in **void main()**
 - Initialization:
 - Start with **$z = \text{vec2}(0)$** , meaning the origin in the complex plane.
 - Iteration:
 - Iterate up to **maxIterations** times.
 - In each iteration, calculate **$zNext = c + \text{complexPow}(z, power)$** .
 - **If the magnitude of $zNext$ exceeds 2, break the loop. This indicates the point escapes the Mandelbrot set. Hint: $Magnitude = \text{length}(zNext)$**
 - Coloration:
 - Output a grayscale color based on whether the point escaped the Mandelbrot set (**$\text{vec4}(color, color, color, 1)$**).

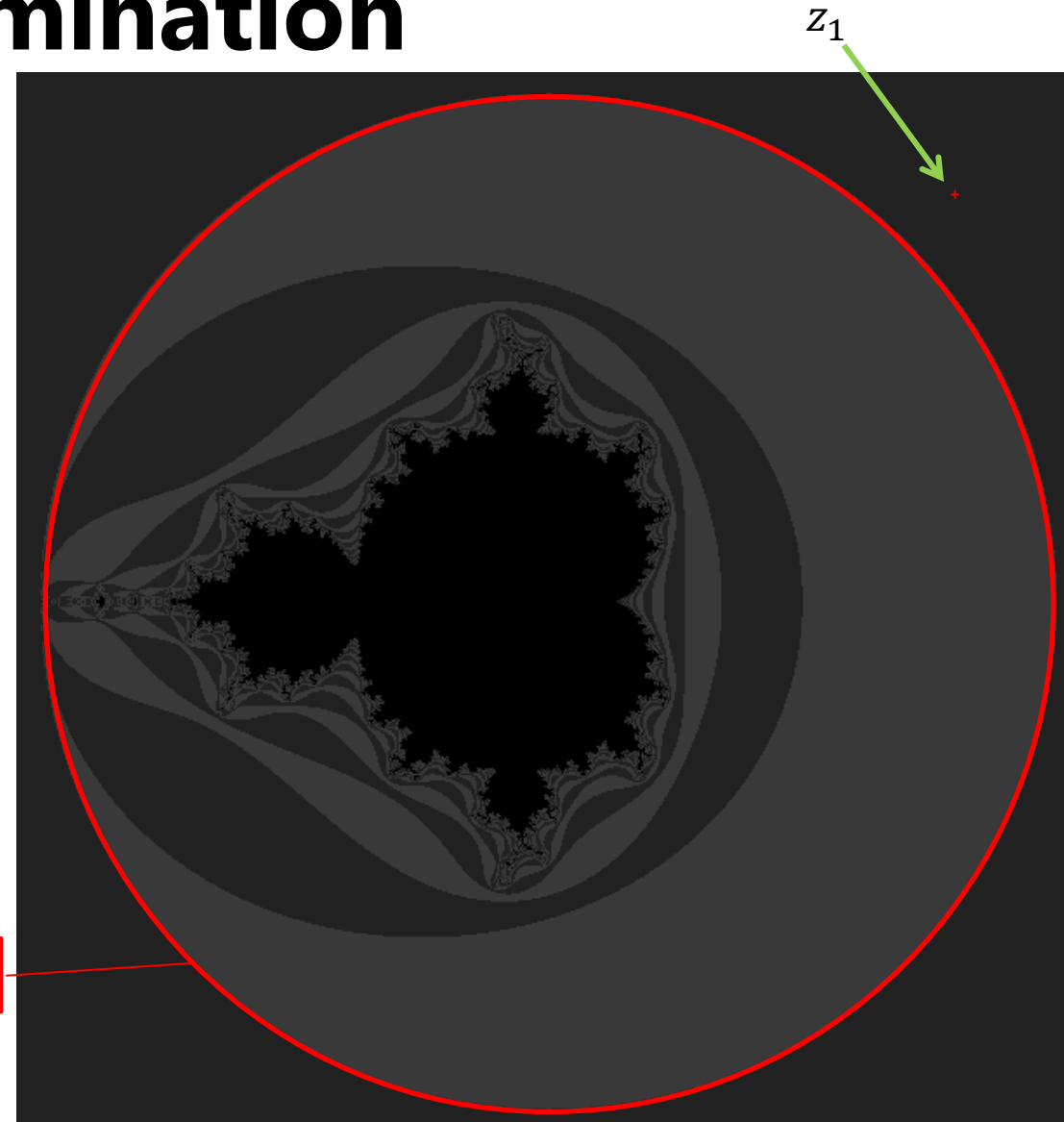
Example – Immediate Termination

$$z_1 = 1.6 + 1.6i$$

$$|z_1| = \sqrt{1.6^2 + 1.6^2} = 2.26 > 2$$

$\Rightarrow \text{STOP!}$

Circle of Radius 2 centered around the origin



Example – Early Termination

$$z_1 = 0.8 + 0.2i$$

$$|z_1| = \sqrt{0.8^2 + 0.2^2} = 0.84 \leq 2 \\ \Rightarrow \text{CONTINUE!}$$

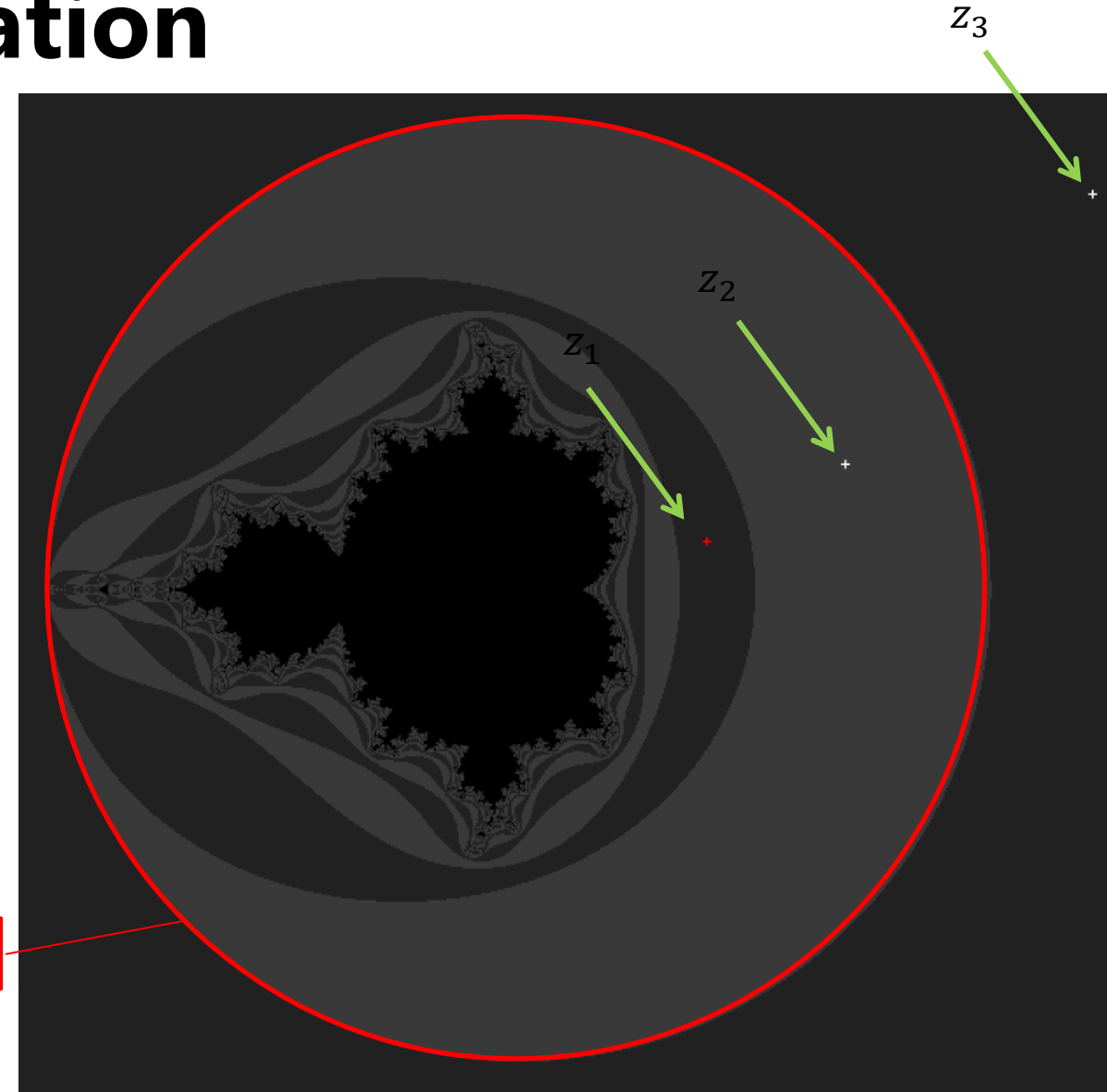
$$z_2 = 1.38 + 0.53i$$

$$|z_2| = \sqrt{1.38^2 + 0.53^2} = 1.47 \leq 2 \\ \Rightarrow \text{CONTINUE!}$$

$$z_3 = 2.42 + 1.67i$$

$$|z_3| = \sqrt{2.42^2 + 1.67^2} = 2.92 > 2 \\ \Rightarrow \text{STOP!}$$

Circle of Radius 2 centered around the origin



Example – Many Iterations

$$\begin{aligned} z_1 &= -0.4 + 0.05i \\ |z_1| &= \sqrt{(-0.4)^2 + 0.05^2} = 0.4 \leq 2 \\ &\Rightarrow \text{CONTINUE!} \end{aligned}$$

...

...

...

$$\begin{aligned} z_{20} &= -0.3 + 0.03i \\ |z_{20}| &= \sqrt{(-0.3)^2 + 0.03^2} = 0.3 \leq 2 \\ \dots &\Rightarrow \text{CONTINUE!} \end{aligned}$$

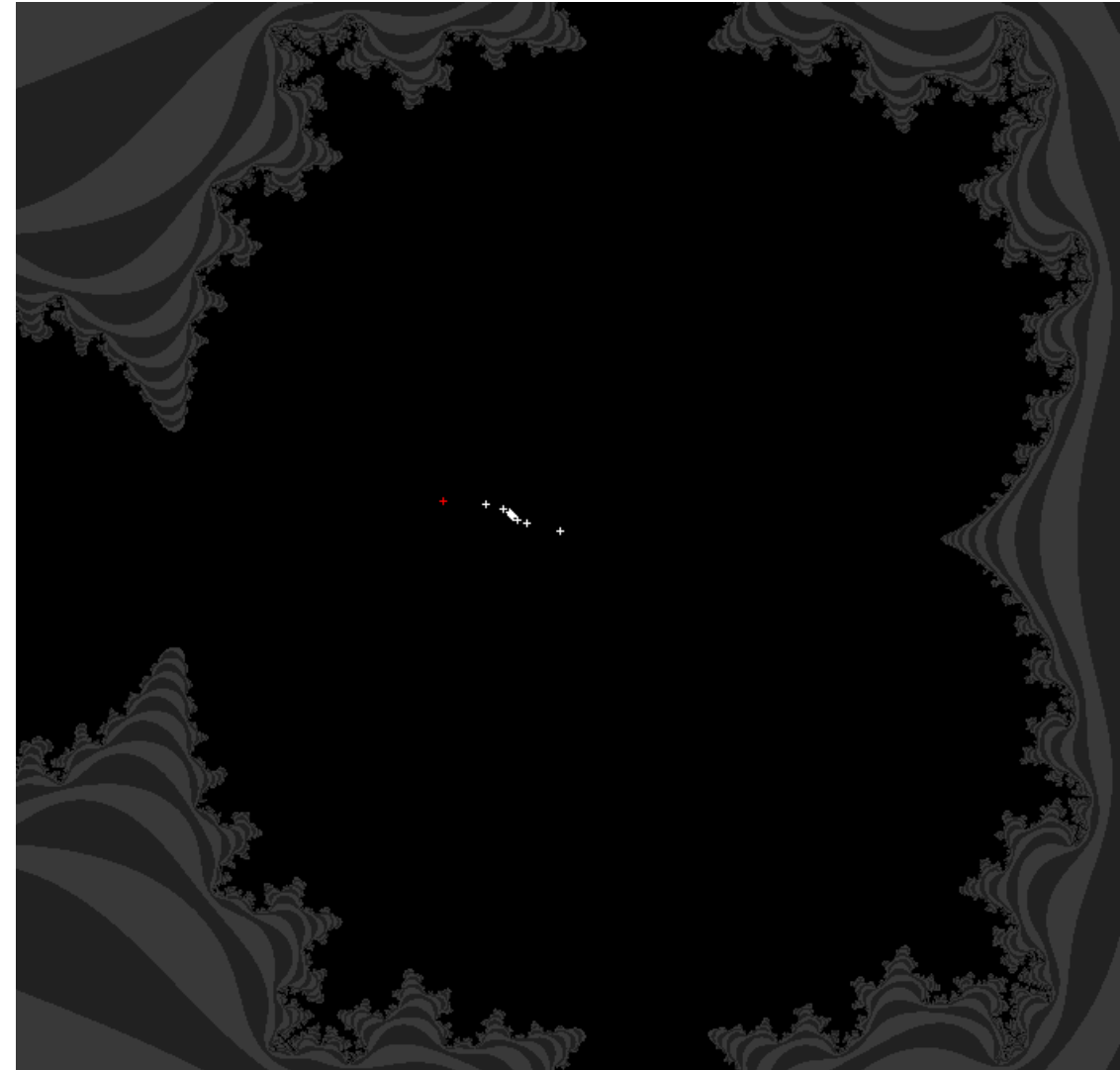
...

...

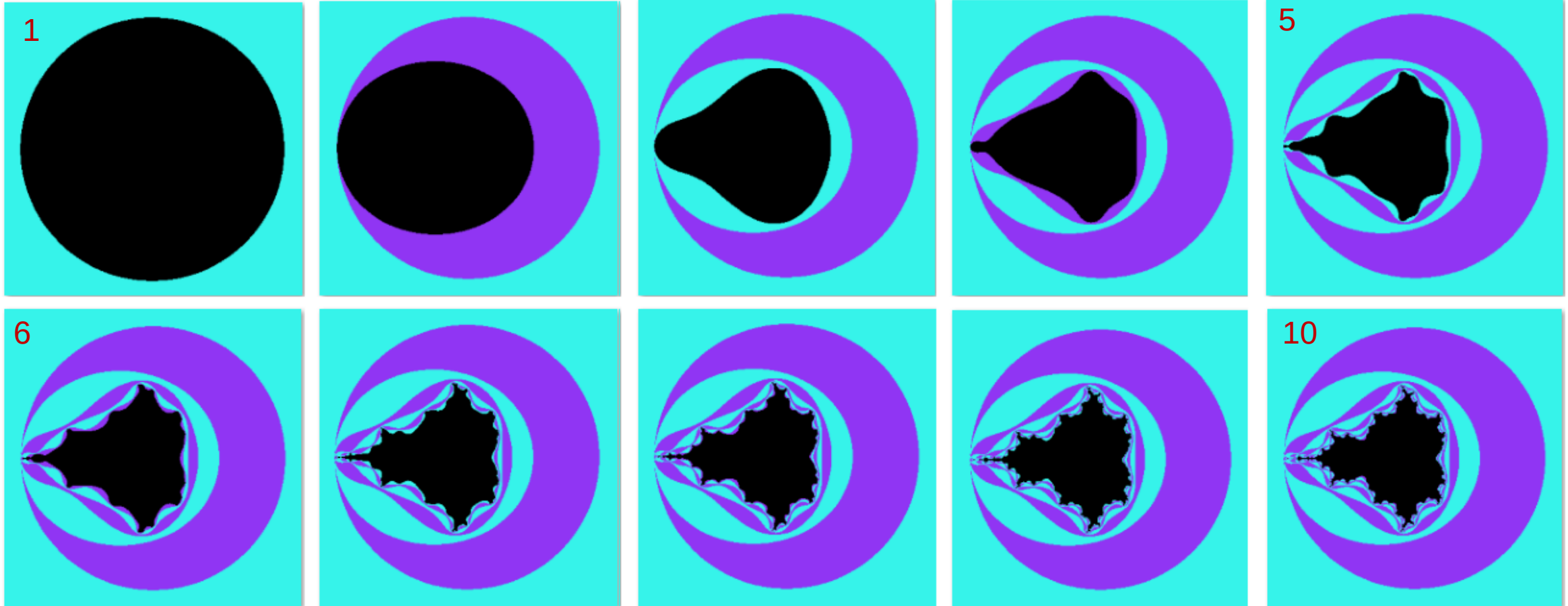
Video that shows “path” of calculated intermediate points:

“Sounds of the Mandelbrot Set” by CodeParade

<https://www.youtube.com/watch?v=GiAj9WW1OfQ>

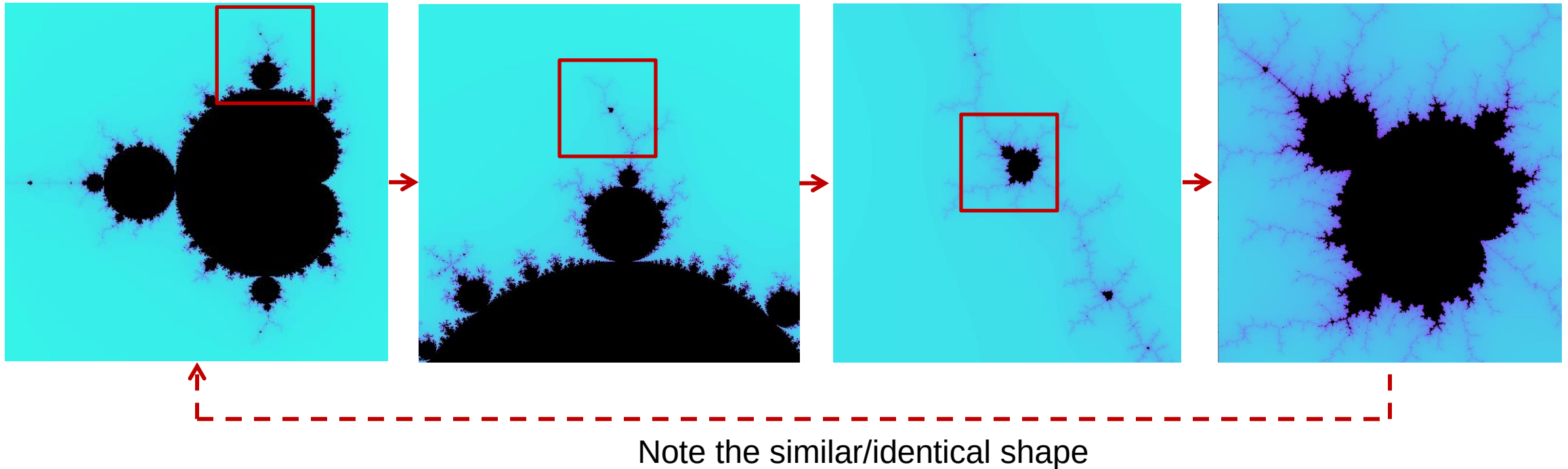


Approximate with finite Number of Iterations



Fractal Sets – Self Similarity

- Patterns re-appear (scaled/rotated) when zooming in



ShaderProgram

- Shader Compilation:
 - Load Shaders:
 - Load the vertex and fragment shader source code from files using the loadFile method.
 - (Implement loadFile yourself, as seen in README)
 - Compile Shaders:
 - Use **glCreateShader** to create shader objects (vertex and fragment).
 - Pass the loaded shader code to **glShaderSource**.
 - Compile the shaders using **glCompileShader**.
 - Optional: Check for compilation errors with **glGetShaderi** and **glGetShaderInfoLog**.
- See Assignment Description for further recommended OpenGL functions and documentation!

ShaderProgram

- Shader Program Creation (Constructor):
 - Create Program:
 - Use **glCreateProgram** to create a shader program object.
 - Attach Shaders:
 - Attach the compiled vertex and fragment shaders to the program with **glAttachShader**.
 - Link Program:
 - Link the shaders together into a complete program using **glLinkProgram**.
 - Optional: Verify the linking success using **glGetProgrami** and **glGetProgramInfoLog**.
 - Use Program:
 - Activate the shader program with **glUseProgram**.
- Cleanup:
 - In the close method, delete the shaders and program using **glDeleteShader** and **glDeleteProgram** to free up resources.
- See Assignment Description for further recommended OpenGL functions and documentation!

ShaderProgram: Used OpenGL Functions

- **glCreateShader(int type):** Creates a shader object of the specified type (GL_VERTEX_SHADER or GL_FRAGMENT_SHADER).
- **glShaderSource(int shader, String source):** Sets the source code in the shader object.
- **glCompileShader(int shader):** Compiles the shader object.
- **glGetShaderi(int shader, int pname):** Retrieves a parameter from a shader object (e.g., compile status).
- **glGetShaderInfoLog(int shader):** Retrieves the compilation log (useful for debugging).
- **glCreateProgram():** Creates a new program object.
- **glAttachShader(int program, int shader):** Attaches a shader object to a program object.
- **glLinkProgram(int program):** Links all attached shader objects.
- **glGetProgrami(int program, int pname):** Retrieves a parameter from a program object (e.g., link status).
- **glUseProgram(int program):** Installs a program object as part of the current rendering state.
- **glDeleteShader(int shader):** Deletes a shader object.
- **glDeleteProgram(int program):** Deletes a program object.

RenderWindow

- Window Initialization:
 - Create GLFW Window:
 - Use **glfwCreateWindow** to create a new window.
 - Context Management:
 - Set the current context to the created window using **glfwMakeContextCurrent**.
 - Initialize OpenGL Capabilities:
 - Initialize OpenGL functions using **GL.createCapabilities**.
 - Set clear color:
 - Use **glClearColor** to set the window's background color based on the selected rendering mode
- Vertex Array and Buffer Setup
 - Generate Vertex Array Object (VAO)
 - Create a VAO with **glGenVertexArrays** and bind it using **glBindVertexArray**.
 - Create Complex Plane Geometry
 - Define a complex plane geometry as a pair of triangles using **createComplexPlane**, which creates a Vertex Buffer Object (VBO) using **glGenBuffers**, **glBindBuffer**, and **glBufferData**.
 - Configure Vertex Attributes
 - Retrieve the position attribute location from the shader using **glGetAttribLocation**.
 - Enable and define the vertex attribute pointer using **glEnableVertexAttribArray** and **glVertexAttribPointer**.

RenderWindow

- Shader Program Setup:
 - Load and Compile Shaders:
 - Create a ShaderProgram instance that loads and compiles the vertex and fragment shaders.
 - Set Projection Matrix:
 - Retrieve the location of the projection uniform using **glGetUniformLocation**.
 - Set up the projection matrix based on the window's properties using **glUniformMatrix4fv**.
- Rendering:
 - Context and Viewport Management:
 - Make the window context current with **glfwMakeContextCurrent** and set the viewport size using **glViewport**.
 - Clear and Draw:
 - Clear the window with **glClear**, apply any post-clear operations using **afterClear**, and render the complex plane using **glDrawArrays**.
 - Read Pixels:
 - Capture the rendered image into a pixel buffer using **glReadPixels**.
 - Swap buffers to display the rendered image using **glfwSwapBuffers**.

RenderWindow: Used GLFW Functions

- `glfwCreateWindow(int width, int height, String title, long monitor, long share):`
 - Creates a new GLFW window.
- `glfwMakeContextCurrent(long window):`
 - Makes the OpenGL or OpenGL ES context of the specified window current.
- `glfwSetWindowSize(long window, int width, int height):`
 - Sets the size of the specified window.
- `glfwSwapBuffers(long window):`
 - Swaps the front and back buffers of the specified window.
- `glfwTerminate():`
 - Terminates GLFW, freeing any resources allocated by it.

RenderWindow: Used OpenGL Functions

- `glClearColor(float red, float green, float blue, float alpha)`: Sets the clear color for the window.
- `glClear(int mask)`: Clears buffers to preset values.
- `glGenVertexArrays()`: Generates a vertex array object name.
- `glBindVertexArray(int array)`: Binds the vertex array object.
- `glGenBuffers()`: Generates buffer object names.
- `glBindBuffer(int target, int buffer)`: Binds a buffer object to a target.
- `glBufferData(int target, FloatBuffer data, int usage)`: Creates and initializes a buffer object's data store.
- `glGetAttribLocation(int program, String name)`: Returns the location of an attribute variable in a program object.
- `glEnableVertexAttribArray(int index)`: Enables a generic vertex attribute array.
- `glVertexAttribPointer(int index, int size, int type, boolean normalized, int stride, long pointer)`: Defines an array of generic vertex attribute data.

RenderWindow: Used OpenGL Functions

- **glGetUniformLocation(int program, String name):** Returns the location of a uniform variable.
- **glUniformMatrix4fv(int location, boolean transpose, FloatBuffer value):** Specifies the value of a uniform matrix variable for the current program object.
- **glViewport(int x, int y, int width, int height):** Sets the viewport.
- **glDrawArrays(int mode, int first, int count):** Renders primitives from array data.
- **glReadPixels(int x, int y, int width, int height, int format, int type, ByteBuffer pixels):** Reads a block of pixels from the framebuffer.
- **glDeleteVertexArrays(int array):** Deletes vertex array objects.
- **glDeleteBuffers(int buffer):** Deletes named buffer objects.
- **glDeleteProgram(int program):** Deletes a program object.

MandelbrotRenderer

- The MandelbrotRenderer class extends RenderWindow and specializes it for rendering Mandelbrot fractals. It verifies that the fractal type is correct, initializes shaders, retrieves uniform locations, and sets the necessary uniform values before rendering. The uniform variables control key aspects of the fractal rendering, such as the power and iteration limit.



Image Source: <https://i.redd.it/05c0zf0xaei31.jpg>

MandelbrotRenderer

- Initialization (init Method):
 - Call to Superclass Initialization:
 - Call the init method of the RenderWindow superclass, passing the vertex and fragment shader file paths ("complexPlane.vert" and "mandelbrot.frag") along with the FractalRenderOptions.
 - Uniform Variable Locations:
 - Retrieve the locations of the uniform variables power, maxIterations, and colorful in the fragment shader using **glGetUniformLocation** and store them in the corresponding instance variables.
- Setting Uniform Variables (afterClear Method):
 - Setting power:
 - Set the value of the power uniform in the shader to the value obtained from options.getPower() using **glUniform1f**.
 - Setting maxIterations:
 - Set the maxIterations uniform in the shader to the number of iterations from options.getIterations() using **glUniform1i**.

MandelbrotRenderer: Used OpenGL Functions

- **glGetUniformLocation(int program, String name):**
 - Retrieves the location of a uniform variable within a shader program. This location is required to set the value of the uniform variable later in the rendering process.
- **glUniform1f(int location, float v0):**
 - Sets the value of a uniform variable at the specified location to a single floating-point value. In this context, it is used to set the power uniform for the Mandelbrot fractal calculations.
- **glUniform1i(int location, int v0):**
 - Sets the value of a uniform variable at the specified location to a single integer value. This is used to set the maxIterations and colorful uniforms in the Mandelbrot rendering process

GpuRenderer

- The GpuRenderer class is responsible for managing the rendering of fractals using OpenGL and GLFW.
- It handles the initialization of the window and rendering contexts, delegates rendering to specialized classes (MandelbrotRenderer), and ensures proper cleanup of resources.
- The main loop in the main method animates the fractals by gradually changing the power parameter and measures the time taken for rendering each frame.
- This class serves as the entry point for GPU-based fractal rendering.

Image Source: <https://i.redd.it/rgs0kvbb83l51.png>



GpuRenderer

- Class Structure and Initialization:
 - Attributes:
 - The GpuRenderer class manages **mandelbrot** (an instance of MandelbrotRenderer)
 - Constructor:
 - The default constructor initializes the GpuRenderer class but doesn't set up anything yet.
 - Initialization (init Method):
 - GLFW Initialization:
 - The method begins by initializing the GLFW library with **glfwInit()**. If initialization fails, it returns false.
 - Set default window hints with **glfwDefaultWindowHints()** and configure window properties (visibility, resizable, OpenGL version).
 - Renderer Setup:
 - Instantiate MandelbrotRenderer.
 - Initialize the renderer with the respective FractalRenderOptions. If it fails, the method returns false.
 - If everything initializes correctly, the method returns true.

GpuRenderer

- Rendering (render Method):
 - The render method takes FractalRenderOptions as input
 - It returns a ByteBuffer containing the rendered image data.
- Resource Management (close Method):
 - Cleanup:
 - The close method ensures proper cleanup by closing the mandelbrot renderer.
 - This is important to free up any allocated resources.
- Main Method (main):
 - Setup and Loop:
 - The main method creates an instance of GpuRenderer within a try-with-resources block to ensure close is called automatically.

GpuRenderer

- Renderer Initialization:
 - Initializes the GpuRenderer with the Mandelbrot options.
- Render Loop:
 - A loop continuously renders the Mandelbrot fractal, measures the time taken for rendering, and prints it in milliseconds.
 - **The power value is gradually increased by 0.01 for the fractal with each loop iteration to animate the fractal.**

GpuRenderer: Used OpenGL Functions

- **glClear(int mask):** Clears the buffer specified by mask (e.g., GL_COLOR_BUFFER_BIT).
- **glViewport(int x, int y, int width, int height):** Sets the viewport, defining the drawable region of the window.
- **glReadPixels(int x, int y, int width, int height, int format, int type, ByteBuffer pixels):** Reads the pixels from the framebuffer and stores them in a ByteBuffer. Used for capturing the rendered image.
- **glUniform1f(int location, float v0):** Sets the value of a uniform variable in the shader to a floating-point value.
- **glUniform1i(int location, int v0):** Sets the value of a uniform variable in the shader to an integer value.

GpuRenderer: Used GLFW Functions

- **glfwInit():** Initializes the GLFW library. Necessary for any subsequent GLFW operations.
- **glfwDefaultWindowHints():** Resets all window hints to their default values.
- **glfwWindowHint(int target, int hint):** Sets hints for the next created window. For example, setting visibility, resizable, and OpenGL profile.
- **glfwCreateWindow(int width, int height, String title, long monitor, long share):** Creates a window with the specified width, height, title, monitor, and shared context.
- **glfwMakeContextCurrent(long window):** Makes the OpenGL context of the specified window current on the calling thread.
- **glfwSetWindowSize(long window, int width, int height):** Sets the size of the specified window.
- **glfwSwapBuffers(long window):** Swaps the front and back buffers, updating the window with the latest rendering.

OOP2 KU

A Layman's Introduction to Fractals and the Bonus Assignment

Session X

Sebastian David Albu, WS 2024

Based on slides from WS 2021 and WS 2022

Institute of Computer Graphics and Knowledge Visualization (CGV)